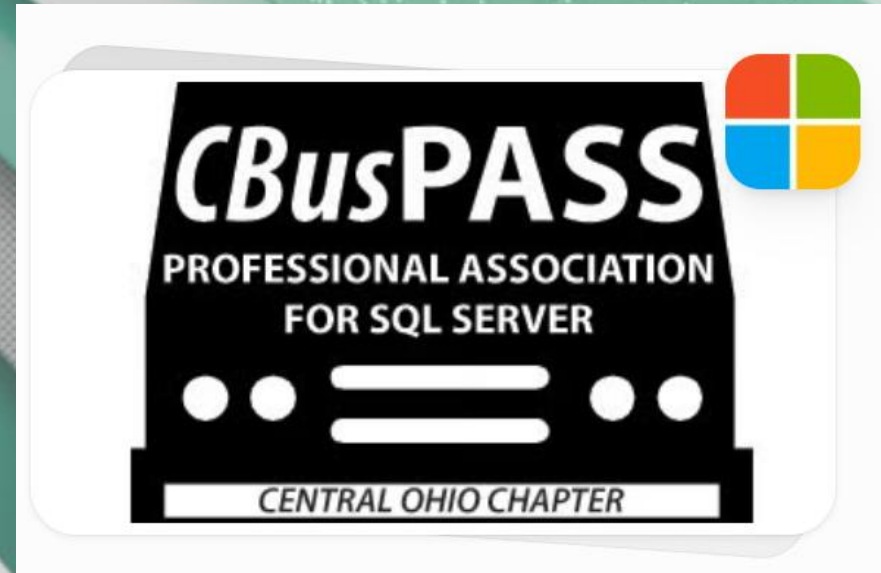


Practical AI for Power BI Developers

Andrew Boerger



Bio

Andrew Boerger

Analytics Manager
AGC Automotive North America

Live & work in Bellefontaine, Ohio

[LinkedIn](#) | [Reddit](#) | [aboerg.dev](#)



Bio 2

Just a user

Fabric Analytics Engineer Associate (DP-600)
Fabric Data Engineer Associate (DP-700)

HackTogether: The Microsoft Fabric Global AI Hack
Best use of Copilot for Microsoft Fabric: Accounts Payable Payment Prediction

Administering Power BI Premium since 2020

Using Fabric since day 1 of Public Preview



Repo



My organization

World leader in automotive glass

North American business unit (~3K employees)

Using Power BI (2018), Premium (2020),
Synapse (2021), Fabric (2023)

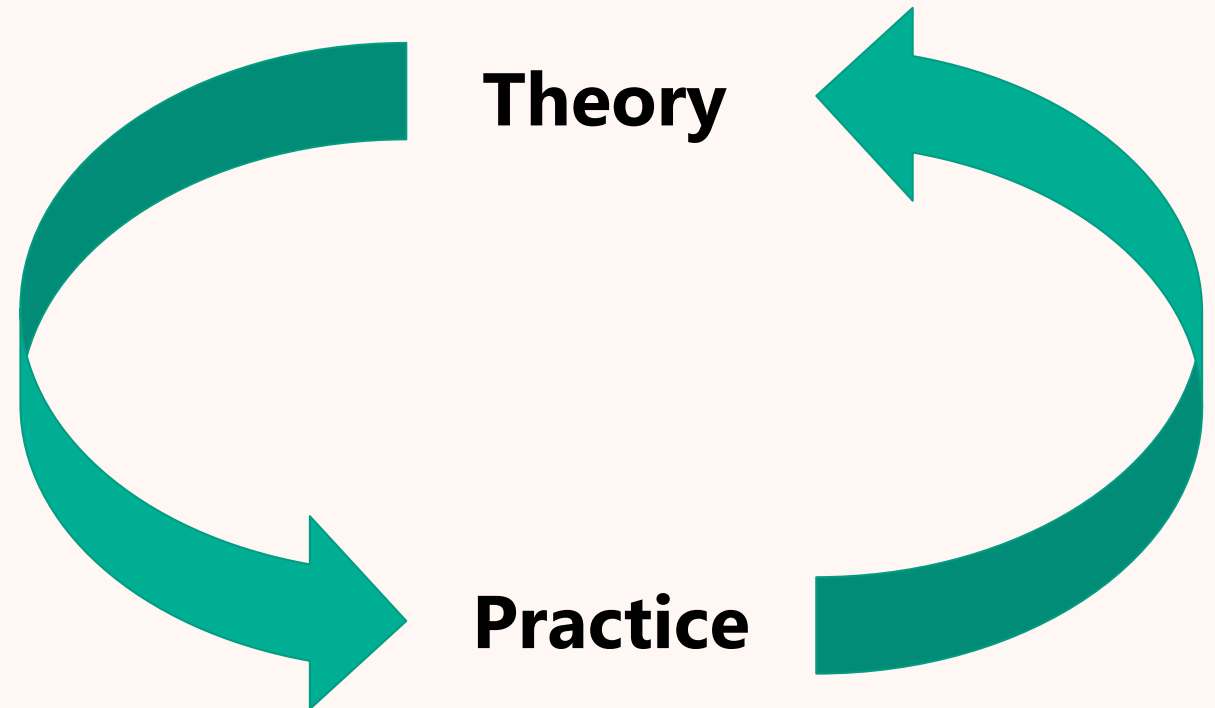


I'm not an "AI expert."

I'm just trying to keep up.

*"Experience teaches nothing. In fact, there is no experience to record without theory... **Without theory there is no learning**... People copy examples and then they wonder what is the trouble. They look at examples and without theory they learn nothing."*

- W. Edwards Deming



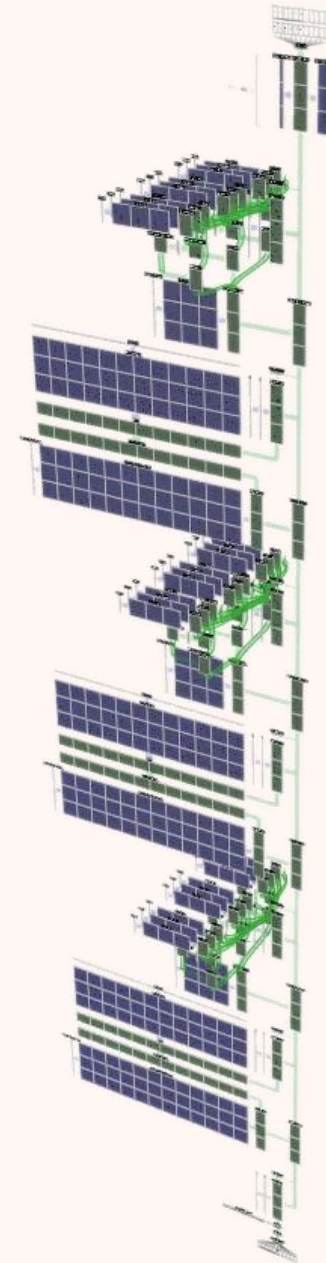
Part 1: Practical AI...

Part 2: ...For Power BI developers

Part 3: Demos!

Part 1:

Practical AI



<https://bbycroft.net/llm>

Large language models
predict the next token.

Wait...what's a token?

Visualize LLM Tokens

GPT2

GPT3.5 / GPT4

LLaMA

The landing zone is where all data from external source systems is landed. Flat files are stored in directories partitioned by ingestion time, ingestion type, and a unique load key.

Tokens: 35 Characters: 181

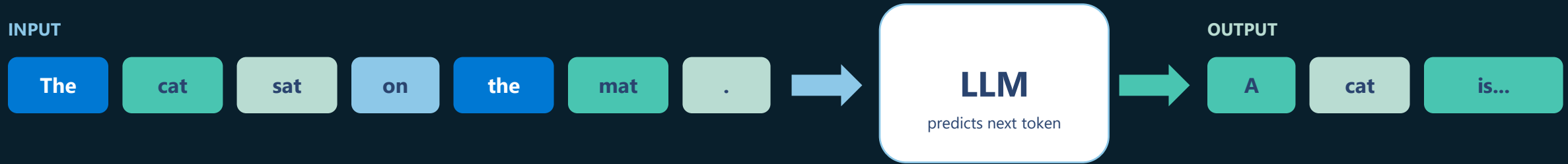
Clear

Show example

The landing zone is where all data from external source
systems is landed . Flat files are stored in directories
partitioned by ingestion time , ingestion type , and a
unique load key .

Visualize it → tokenvisualizer.netlify.app

The context window



Every input + output is tokenized first. 1 token $\approx \frac{3}{4}$ of a word.

Context window = model's short-term memory (measured in tokens)

FRONTIER MODELS

200K – 1M+

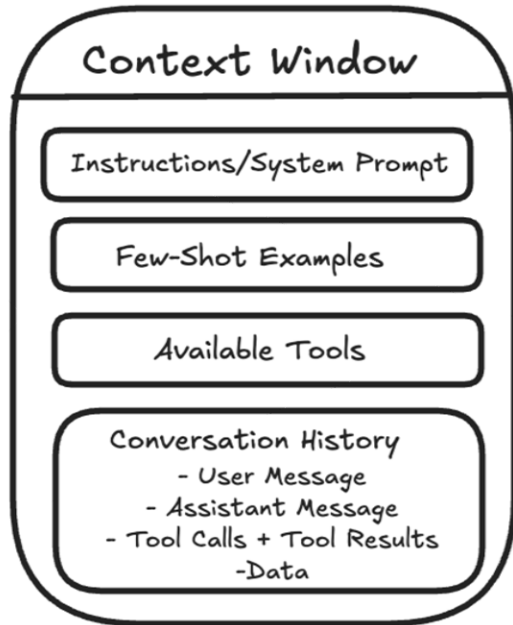
tokens (e.g., GPT-5.5, Claude Sonnet 4.6, Gemini 3.1)

CHEAPER, FASTER MODELS

128K – 200K

tokens (e.g., GPT-5 mini, Claude Haiku, Gemini Flash)

The context window



THE MECHANICS

No persistent state between turns

The model loads fresh every turn. Weights don't change. Nothing carries over.

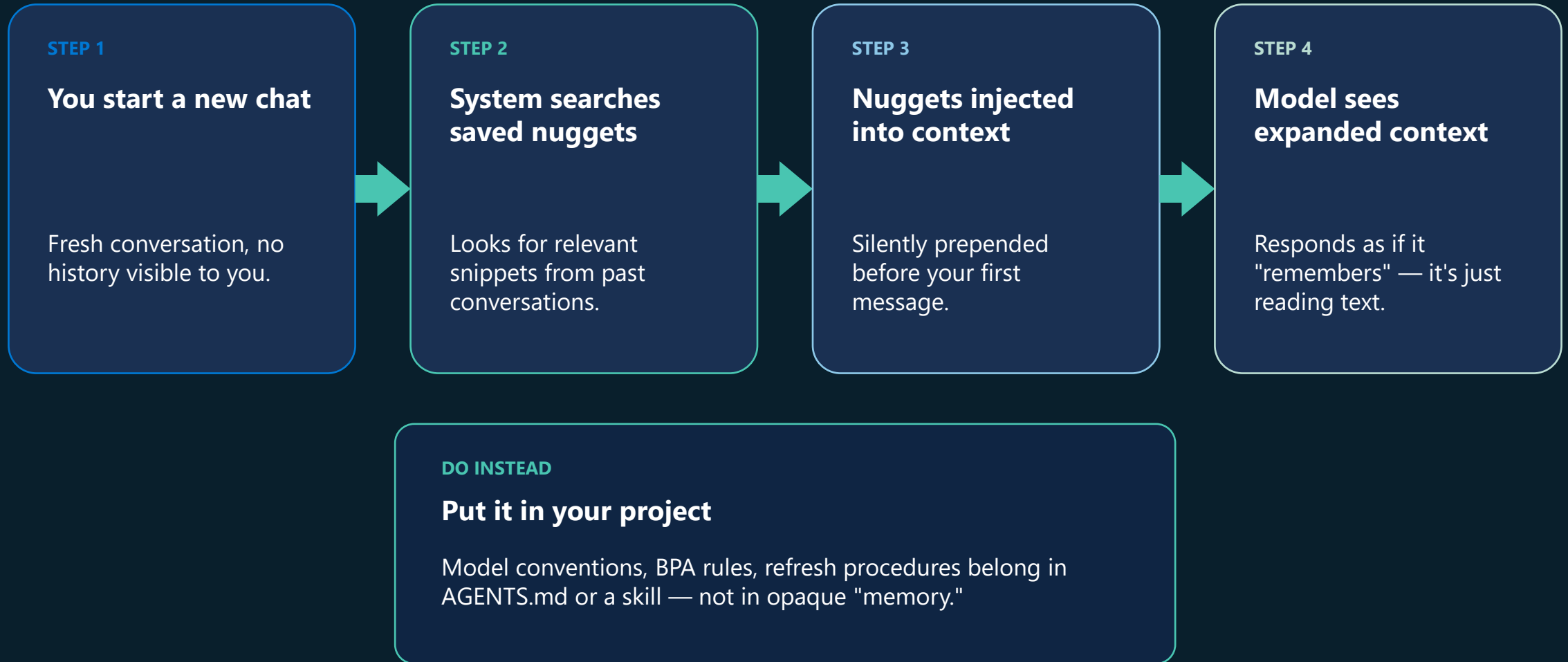
"Context" is the entire chat transcript replayed

Context window is finite

WHY THIS MATTERS

There is no "the agent remembers." Anything it knows about you, your repo, or your model lives in text — replayed every turn. Context should live in a file you control: AGENTS.md, a skill, the PBIP repo. Not in conversation history.

"Memory" is inserted text



The system prompt

ANATOMY OF A SYSTEM PROMPT

PERSONA

You are Claude, a helpful AI...

TOOLS

<bash>, <file_read>, <web_search>...

SAFETY RULES

Refuse harmful requests, cite sources

FORMATTING

Use markdown, prefer code fences

DEFAULT BEHAVIOR

Ask clarifying questions, be concise

WHY IT MATTERS

ALWAYS-ON

You don't see it. It's always there.

Sometimes thousands of tokens of context loaded before your first message even arrives.

INVISIBLE INFLUENCE

It shapes every response.

Persona, tools, safety, formatting, defaults — all set before the model considers your question.

THE TAKEAWAY

The raw model is not what you talk to.

The harness shapes everything. Treat the system prompt as part of the product.

Prompt engineering to context engineering

OLD: PROMPT ENGINEERING

"You are a senior engineer who..."

Doesn't move the needle anymore. The model already knows how to be helpful.

NEW: CONTEXT ENGINEERING

Get the right information into the context window.

Coding standards, project layout, prior decisions, examples. Get them out of your head, into files the agent can read.

**A perfect prompt with no context
loses to a casual prompt with great context.**

Markdown

WHY MARKDOWN?

Models speak text.

Project setup

Build commands

- `npm run build` — production bundle
- `npm test` — run unit tests

Things never to touch

- `/legacy/` — do not modify
- generated `\*.tmdl` files

Clean, structured, plain text.

01

Almost every agent-readable file in your project is .md

02

READMEs, agent instructions, skills, prompts ... most assume markdown

03

Diff-friendly, human-friendly, model-friendly

You will write a lot of it.

AGENTS.md / CLAUDE.md / copilot-instructions.md

THE SHIFT

README.md is for humans. AGENTS.md is for the agent.

WHAT IT IS

A markdown file at the project root

Tells the agent how to behave in your Power BI repo — the same way you'd brief a new analyst joining the team.

WHAT GOES IN IT

Conventions and guardrails

PBIP project layout, TMDL naming conventions, BPA rule sets, and the measures never to touch.

WHO READS IT

Every major agent

AGENTS.md is an open standard. CLAUDE.md and copilot-instructions.md are tool-specific equivalents.

IF YOU DO ONE THING FROM THIS TALK

Add an AGENTS.md to your project today.

Layered context: global / project / local

PROJECT MEMORY — THREE FILES, ONE PRECEDENCE ORDER

GLOBAL

`~/.claude/CLAUDE.md`

Your personal preferences

Applies to every Power BI project you touch. "Always write DAX with explicit table prefixes." "Use kebab-case for TMDL filenames."

PROJECT

`AGENTS.md`

Team standards

Committed alongside your PBIP. Every analyst — and every agent — reads the same model conventions.

LOCAL

`.local files`

Personal sandbox

Gitignored. Path overrides, machine-specific tweaks that don't belong in the shared repo.

PRECEDENCE

Team standards beat personal preferences

Agent reads all three, but project rules override your globals.

NOTHING NEW HERE

Same pattern as `.gitignore`, ESLint config, `.editorconfig`

We've been doing layered config for decades. Apply it to AI.

Skills: SOPs as markdown

ANATOMY OF A SKILL

```
bpa-rules/  
├── SKILL.md  
│   title + description load eagerly  
│   full body loads when relevant  
├── bpa-syntax.md  
├── rule-examples/  
└── te-cli/
```

WHAT'S IN SKILL.MD

"When asked to **author a BPA rule, follow these steps. Here are the rules. Here are examples."**

An SOP for the agent — the same kind of doc you'd hand a new hire on day one.

PROGRESSIVE DISCLOSURE

Loads on demand

Title and description sit in the agent's context. The full body only loads when the task matches.

OPEN FORMAT

Anyone can write one

Anthropic published the spec. Claude, Cursor, and most coding agents now support skills.

READ OTHERS'

Best-practice in disguise

Other people's skills double as well-edited guides for how a task should actually be done.

MCP: Model Context Protocol

ONE PROTOCOL, MANY TOOLS: AN OPEN STANDARD FROM ANTHROPIC, LATE 2024

HOW IT WORKS



THE PROBLEM IT SOLVES

Replaces a thousand custom integrations

One client-server model instead of glue code for every tool.

MCP SERVERS PROVIDE "TOOLS" FOR AGENTS

Apply BPA. Read TMDL. Refresh a model.

Power BI capabilities surfaced to the agent over one simple, text-based interface.

WHY IT MATTERS

Agents discover tools without code

tabular-editor, pbi-desktop, fabric-cli, pbip — plug in and go.

THE BIG IDEA

MCP is to AI agents what USB-C is to devices — one connector, every peripheral.

MCP vs Skills

TWO WAYS TO TEACH YOUR AGENT

MCP

Live tools, runtime calls

- **Live tools** —> call a running service
- **Always loaded** in context window
- **Two-way state** —> read AND write back
- Needs an MCP server running locally or remote
- **Example:** pbi-desktop can query your local model

SKILLS

Markdown SOPs, static files

- **Static instructions** —> markdown + files
- **Progressive disclosure** —> body loads on demand
- **One-way knowledge** —> agent reads, decides
- Just files in a folder
- **Example:** bpa-rules/ teaches BPA syntax

WHERE MCP HURTS

Token bloat

Every tool description sits in context — even unused ones. **A big MCP server can eat thousands of tokens before you've done anything.**

Hooks: deterministic validations

AUTOMATIC VALIDATION AFTER EVERY EDIT — TIGHTENS THE FEEDBACK LOOP

THE FEEDBACK LOOP

- 1 Agent edits TMDL file**
`model.tmdl`
- 2 Hook fires automatically**
`PostToolUse`
- 3 Validator runs**
`tmdl-validate ./model.tmdl`
- 4 Result fed back to agent**
`✓ valid | ✗ line 42: syntax error`

WHY IT MATTERS

Instant feedback on validity

No more "that should work" guesses — the agent sees pass/fail before it moves on.

WHEN IT'S CRITICAL

Newer or niche tools

TMDL, Bicep, custom DSLs — anything the model hasn't seen much in training.

THE EFFECT

Fewer "actually that didn't work" rounds

The agent self-corrects in one shot instead of three. Real time saved.

THE BIG IDEA

A hook turns "I think this works" into "the validator says yes."

Plugins: everything you need, together

ONE INSTALL = A WHOLE DOMAIN OF EXPERTISE

ONE PLUGIN BUNDLES...

Skills

Markdown SOPs the agent reads

MCP servers

Tool integrations for external APIs

Hooks

Validators run after edits

Commands

Slash-prefixed shortcuts

Sub-agents

Specialized helpers it can spawn

THE PROMISE

Install one thing

The agent gains a whole domain of expertise — no per-component setup.

PORTABLE STANDARD

Claude Code, Codex, Copilot CLI

All three CLIs support the same plugin pattern. Write once, run across.

HOW EXPERTS SHARE NOW

Not slide decks — plugins

One practitioner writes the skill once, the rest of the community installs it.

THE PUNCHLINE

Knowledge transfer between practitioners now happens through plugins, not presentations.

Part 2:

For Power BI developers



Power BI Project (PBIP) format

OLD

.pbix

Zipped binary format. Terrible for diffs and merges. Source control can't see inside.

NEW

.pbip

Report and model as text files: TMDL, PBIR JSON. Designed for source control. Pairs with Fabric Git integration.

WHY IT MATTERS

Without Git, you can't collaborate confidently, with your coworkers OR agents.

Fabric workspaces + Git integration

WORKSPACE ↔ REPO

Two-way Git sync

Connect a workspace to an Azure DevOps or GitHub branch. Changes flow both ways.

VERSIONED CODE

Lakehouse + warehouse + notebooks

All become reviewable, diffable artifacts under source control.

WITH PBIP

End-to-end source control

Reports, semantic models, and data items in one Git-tracked solution.

WHY IT MATTERS

The substrate for agents

Without this, agents have nothing to read, write, or review against.

data-goblin / power-bi-agentic-development

A PLUGIN MARKETPLACE FOR POWER BI WORK — OPEN SOURCE, GPL-3.0

`tabular-editor`

Tabular Editor expert

BPA rules, C# scripting (TOM, LINQ), macros, CLI invocation, TE3 config, doc search.

`pbip`

TMDL & PBIP authoring

Edit semantic models as code — TMDL syntax, PBIP project structure, conventions.

`pbi-desktop`

Connect to Desktop's local AS

Talks to the local Analysis Services instance via TOM and ADOMD.NET.

`fabric-cli-plugin`

Fabric workspace operations

Separate plugin for everything that lives in Fabric — workspaces, items, deployment.

INSTALL IN

Claude Code • Codex • Gemini CLI • GitHub Copilot

PQL.Assert: DAX unit testing for semantic models

github.com/clientfirsttech/PQL.Assert

EXAMPLE • `TotalSales.dev.test.dax`

```
// Test: TotalSales returns expected value for FY2024
```

```
EVALUATE PQL_Assert.Equals(  
    [Total Sales (FY24)], // actual  
    12500000,             // expected  
    "TotalSales.FY24"     // test name  
)
```

```
// → returns Pass / Fail row
```

```
// → CI pipeline fails on any test failure
```

NAMING CONVENTION

`[name].[env].test(s)`

DEV, TEST, PROD, or ANY. Pipeline picks tests by environment automatically.

DELIVERY

Native DAX, MIT licensed

Drops into any semantic model. Tests live alongside measures in TMDL.

AUTHORING

Custom Copilot agent

"Power BI Semantic Model Test Specialist" generates tests from plain English.

TAKEAWAY

Create unit tests in your semantic models. Run them locally, or remotely with a simple EVALUATE call.

Part 3:

Demos!



Thank you.

Questions?